

Session 10 : Ombres en Temps Réel — le Shadow Mapping

💡 Objectifs de la session

Comprendre comment le GPU calcule les ombres portées en temps réel. La **Session 5** a posé la question — “comment le GPU sait-il qu’un fragment est derrière un autre vu depuis la lumière ? (Dans l’ombre)” — et a promis la réponse en Session 6. Le baking (Session 6) l’a esquivée en précalculant les ombres statiques dans la lightmap. Aujourd’hui on y revient : le **Shadow Mapping** est LA technique universelle du temps réel, utilisée par Three.js, Unity, Unreal, Godot, et jusqu’aux moteurs custom. On va voir le principe, les artefacts (acne, peter panning), les filtres d’adoucissement (PCF, PCSS), les variantes pour grandes scènes (CSM) et pour lumières ponctuelles (cube shadow maps), puis l’implémentation pratique en Three.js avec une démo interactive, et un aperçu des réglages dans Godot et Unity.

💡 Rappel : ce qu'on sait déjà

- **Session 5** : l’éclairage direct calcule $\vec{N} \cdot \vec{L}$ par fragment. Une surface dos à la lumière reçoit 0 — mais ce n’est pas une ombre, c’est juste l’absence d’éclairage direct. Une **ombre portée** apparaît quand un objet **bloque** la lumière d’un autre.
- **Session 6** : les lightmaps précalculent les ombres dans une texture (qualité ray tracing, coût nul), mais uniquement pour la géométrie **statique**, un moteur de rendu a calculé les ombres au préalable. Un personnage qui marche n’a pas d’ombre baked — il va nous falloir trouver des solutions pour le temps réel.
- **Session 8** : On a abordé les **render target** qui sont des textures dans lesquelles le GPU rend au lieu d’afficher. Le shadow mapping est réutilise cette idée.

Partie 1 : Pourquoi les Ombres ?

Le rôle perceptif des ombres

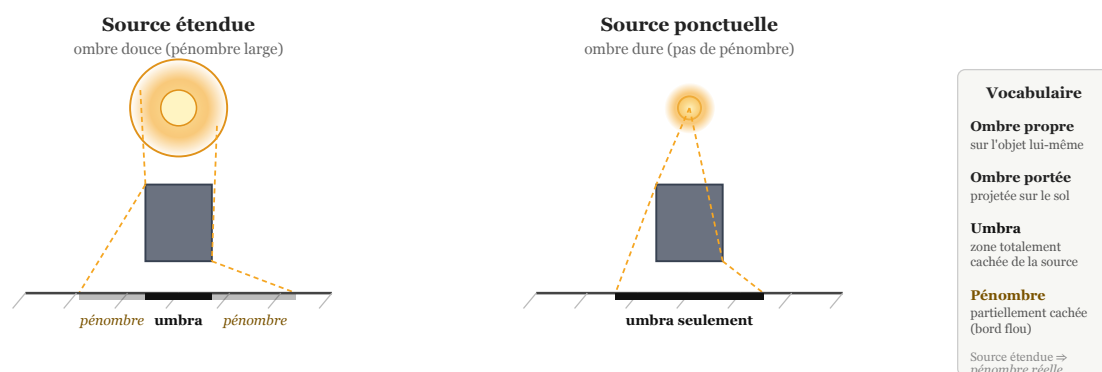
Une scène sans ombres paraît **plate et flottante** : on ne sait pas si un objet touche le sol, ni à quelle distance il se trouve. Les ombres apportent trois informations critiques :

- **Contact** : une ombre attachée à la base d’un objet confirme qu’il est posé. Sans elle, l’objet semble flotter.
- **Profondeur et position** : la longueur et la direction de l’ombre révèlent la position de l’objet dans l’espace et la hauteur de la source.
- **Échelle** : une ombre longue indique une lumière rasante (lever/coucher), une ombre courte indique une lumière au zénith.

💬 Ombre propre vs ombre portée

- **Ombre propre** (self-shadow) : les parties de l’objet non éclairées — déjà gérées par $\max(\vec{N} \cdot \vec{L}, 0)$ (Session 5). C’est le côté sombre de la sphère.
- **Ombre portée** (cast shadow) : l’ombre que l’objet **projette** sur d’autres surfaces. C’est celle-ci qui demande le shadow mapping — le GPU ne peut pas la déduire de la seule normale du receveur.

1. Anatomie d'une ombre



Anatomie d'une ombre : umbra (cœur plein) et pénombre (bord adouci). Une source ponctuelle idéale ne produit que de l'umbra.

Figure 1: Anatomie d'une ombre. L'**umbra** est la zone totalement cachée de la source ; la **pénombre** est la zone partiellement cachée, qui produit le bord flou. Une source ponctuelle idéale (à droite) ne produit que de l'umbra — d'où l'aspect “tranché” des ombres temps réel non filtrées.

Umbra et pénombre

- **Umbra** : région d'où la source est **entièrement** masquée par le bloqueur. Aucune lumière n'arrive — c'est le cœur plein de l'ombre.
- **Pénombre** : région d'où la source n'est que **partiellement** masquée. Une partie des rayons arrive — c'est le bord adouci.
- **Source étendue** : un soleil ou une ampoule a une **taille** (un disque, pas un point). C'est ce qui crée la pénombre. Plus la source est grande (ou proche), plus la pénombre est large.

💡 Conséquence pour le temps réel

Le shadow mapping de base modélise une **source ponctuelle** — il ne produit que de l'umbra (bord net). Pour retrouver la pénombre d'une source étendue, il faut **filtrer** (Partie 5 : PCF, PCSS). C'est la principale différence entre une ombre “jeu vidéo ancienne génération” (nette, tranchée) et une ombre moderne (douce, réaliste).

Partie 2 : Le Shadow Mapping — Principe

L'idée centrale

“Ce qui est visible depuis la lumière est éclairé ; ce qui est masqué vu depuis la lumière est dans l'ombre.”

Le GPU ne “sait” pas qu'un fragment est derrière un objet. Mais il sait **mesurer la profondeur**. Si on rend la scène **depuis la lumière** et qu'on garde la profondeur de chaque pixel, on obtient une carte de “ce que la lumière voit”. Ensuite, pendant le rendu caméra, pour chaque fragment on regarde sa profondeur **vue depuis la lumière** : si elle est **plus grande** que la valeur stockée, c'est qu'un autre objet est plus proche de la lumière à cet endroit — donc le fragment est dans l'ombre.

1. Les deux passes

Le rendu en deux passes

1. **Passe 1 — Shadow map** : on place la caméra à la position de la lumière, on oriente vers la scène, on rend **uniquement la profondeur** (pas la couleur) dans une texture — la **shadow map**. C'est un render target (Session 8) au format depth.
2. **Passe 2 — Rendu final** : on rend la scène depuis la caméra du jeu. Pour chaque fragment, on calcule sa position dans l'espace de la lumière, on échantillonne la shadow map à cet endroit, et on compare les profondeurs.

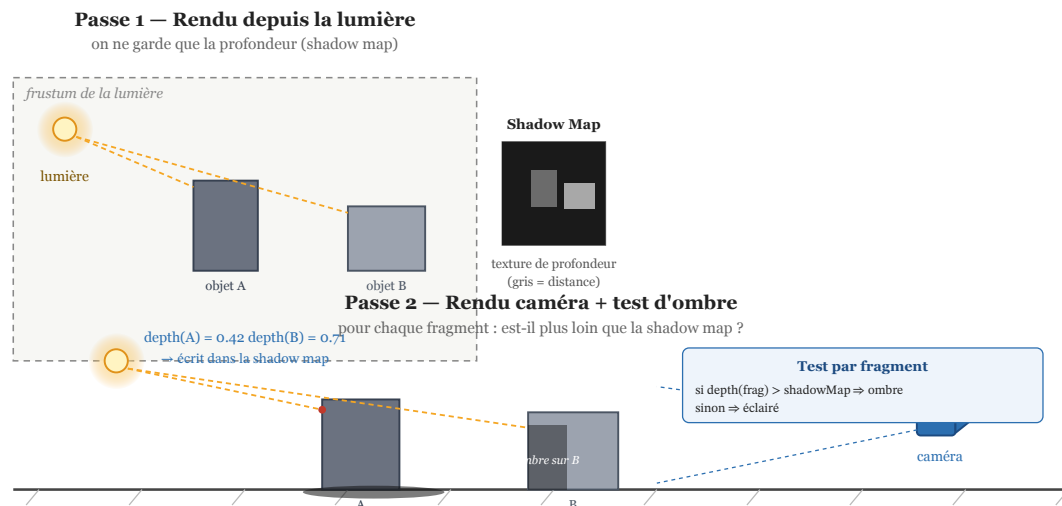


Figure 2: Le shadow mapping en deux passes. Passe 1 : la scène est rendue depuis la lumière, on ne garde que la profondeur (gris = distance) dans la shadow map. Passe 2 : depuis la caméra, chaque fragment est re-projeté dans l'espace lumière ; si sa profondeur dépasse celle stockée dans la shadow map, il est dans l'ombre.

2. La shadow map — une texture de profondeur

Format et contenu

La shadow map est tout simplement le **depth buffer** (Session 3) d'un rendu fait depuis la lumière — sauvegardé dans une texture échantillonnable. Chaque texel contient la profondeur du fragment le plus proche de la lumière à cet endroit.

- **Résolution** : typiquement 1024^2 à 4096^2 . Plus grand = ombres plus nettes, mais plus de VRAM et de coût de rendu.
- **Format** : depth texture (souvent 16 ou 24 bits). 24 bits est le standard — 16 bits provoque de l'acne (Partie 4) par manque de précision.
- **Clear** : initialisée à 1.0 (profondeur maximale) avant la passe 1, comme tout depth buffer.

3. Les coordonnées d'ombre — projeter dans l'espace lumière

Le principe

La shadow map est dans l'espace de la lumière. Le fragment est dans l'espace monde. Pour comparer, on reprojette p_{world} dans l'espace de la lumière — c'est le même pipeline que celui qui place un vertex à l'écran (Session 2), mais avec la **caméra de la lumière** :

$$p_{\text{world}} \xrightarrow{V_L} p_{\text{vue L}} \xrightarrow{P_L} p_{\text{clip L}} \xrightarrow{\div w} p_{\text{NDC L}} \xrightarrow{\text{remap}} (s_x, s_y, s_z)$$

On obtient les **shadow coordinates** (s_x, s_y, s_z) , toutes dans $[0, 1]$:

- (s_x, s_y) : **quel pixel de la shadow map regarder** (UV).
- s_z : **à quelle profondeur est le fragment vu depuis la lumière**.

Le test : si la shadow map à (s_x, s_y) contient une profondeur **plus petite** que $s_z \rightarrow$ un autre objet est plus proche de la lumière $\rightarrow$ le fragment est dans l'ombre.

💡 On compare des profondeurs normalisées, pas des distances

s_z est le z NDC de la lumière (après perspective divide), pas la distance brute |lumière — fragment|. La shadow map stocke la même grandeur. Il faut comparer dans le **même espace**.

💡 En pratique

Tout se précalcule en une matrice : $S = \text{remap} \times P_L \times V_L$. Le shader fait $s = S \times p_{\text{world}}$, puis la perspective divide : $(s_x/s_w, s_y/s_w, s_z/s_w)$. Pour le soleil (orthographique), $w = 1$ donc la divide ne change rien — mais le même shader gère tous les types de lumière.

S est recalculée quand la lumière bouge ou que sa projection change (ex. CSM qui suit la caméra). Pour un soleil statique, on la calcule une seule fois.

Partie 3 : Le Shader d'Ombre

Fragment shader — test d'ombre basique.

```
uniform sampler2D uShadowMap; // la depth texture de la passe 1
uniform mat4 uShadowMatrix; // S = remap * P_light * V_light
in vec3 vWorldPos;

float shadowTest(vec3 worldPos) {
    // 1. Coordonnées d'ombre du fragment
    vec4 s = uShadowMatrix * vec4(worldPos, 1.0);
    s.xyz /= s.w; // perspective divide
    // s.xyz est maintenant dans [0,1] si le fragment est dans le frustum de la
    lumière

    // 2. Hors du frustum lumière ? Pas d'ombre (la lumière ne "voit" pas cet
    endroit)
    if (s.x < 0.0 || s.x > 1.0 || s.y < 0.0 || s.y > 1.0 || s.z > 1.0)
        return 1.0; // éclairé

    // 3. Comparaison des profondeurs
    float storedDepth = texture(uShadowMap, s.xy).r;
    float currentDepth = s.z;

    // 4. Test : si le fragment est plus loin que ce que la lumière voit → ombre
    return (currentDepth > storedDepth) ? 0.0 : 1.0; // 0 = ombré, 1 = éclairé
}
```

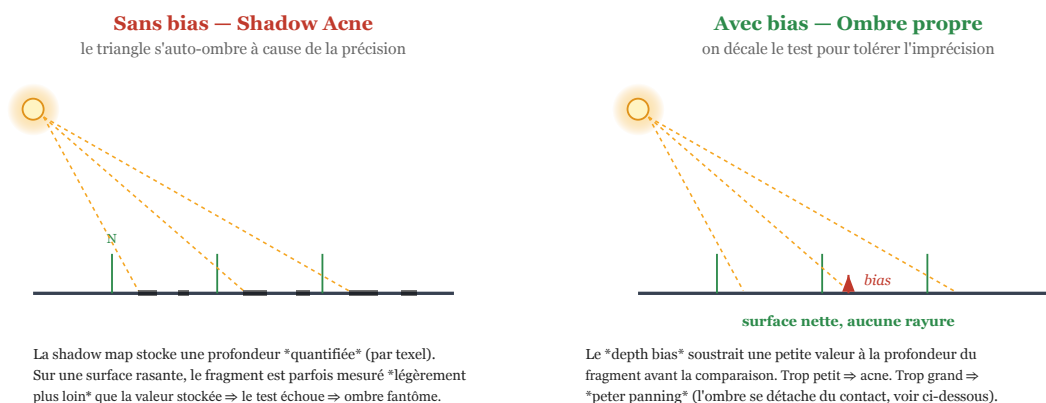
Le résultat (0 ou 1) **multiplie** la contribution de la lumière dans le shader d'éclairage (Session 5) : $\text{color} = \text{diffuse} * (\text{NdotL} * \text{shadow} + \text{ambient})$. Sans cette multiplication, l'ombre n'existe pas — le shadow mapping n'est qu'un **masque**.

Le coût du shadow mapping

- **Passe 1** : un rendu complet de la scène (sans couleur, sans shader complexe — juste la profondeur). Coût GPU proportionnel au nombre de triangles et à la résolution de la shadow map.
- **Passe 2** : un échantillonnage de texture supplémentaire + une comparaison par fragment et par lumière qui projette une ombre.
- **Total** : ajouter une lumière avec ombre coûte **environ un rendu de scène supplémentaire**. C'est pourquoi on limite drastiquement le nombre de lumières avec ombres (1 à 4 en général, contre des dizaines sans ombre).

Partie 4 : Les Artefacts — Acne et Peter Panning

1. Le shadow acne



Le shadow acne : des rayures sombres apparaissent sur les surfaces éclairées en biais. Le bias corrige ce défaut mais peut créer le peter panning.

Figure 3: Le shadow acne : des rayures sombres apparaissent sur les surfaces éclairées en biais. La cause est la **quantification** de la shadow map — chaque texel couvre une zone de la surface, et la profondeur stockée est une valeur moyenne. Un fragment légèrement en arrière de cette valeur échoue le test et devient “ombré” par lui-même.

La cause exacte

La shadow map a une **résolution finie** : un texel couvre plusieurs fragments de la surface. La profondeur stockée est celle du fragment le plus proche **au centre** du texel. Pour un fragment voisin sur une surface en biais, la profondeur réelle diffère légèrement — parfois elle est **juste au-dessus** de la valeur stockée, et le test $\text{currentDepth} > \text{storedDepth}$ réussit à tort. Le fragment s'auto-ombre. C'est d'autant pire que la surface est **rasante** par rapport à la lumière (la profondeur varie vite sur la surface).

La solution : le depth bias

On **décalle** la profondeur du fragment avant la comparaison :

```
float bias = 0.005;
return (currentDepth - bias > storedDepth) ? 0.0 : 1.0;
```

Le bias compense l'imprécision de la shadow map. Trop petit → l'acne persiste. Trop grand → l'ombre se **détache** du contact (peter panning, ci-dessous).

Il existe deux formes de bias :

- **Constant bias** : une valeur fixe soustraite à currentDepth. Simple mais inadapté (la bonne valeur dépend de l'angle de la surface).
- **Slope-scaled bias** : le bias augmente avec la pente de la surface par rapport à la lumière ($\frac{1}{\max(\vec{N} \cdot \vec{L}, \epsilon)}$). Les surfaces rasantes (où l'acne est pire) reçoivent un plus grand bias. C'est le réglage standard des moteurs.

2. Le peter panning

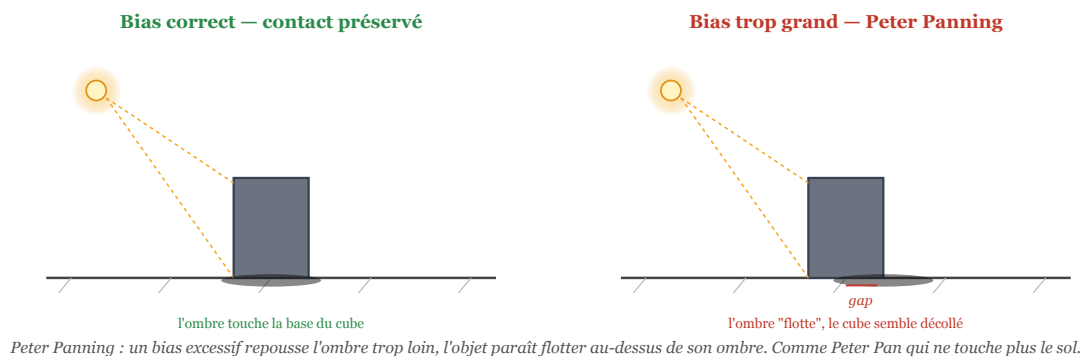


Figure 4: Peter Panning : un bias trop grand repousse l'ombre trop loin de l'objet, qui semble flotter au-dessus de son ombre — comme Peter Pan qui ne touche plus le sol. Le contact visuel entre l'objet et le sol est perdu.

⚠ Le compromis bias

Le bias est un **compromis** permanent :

- **Trop petit** → acne (rayures sur les surfaces éclairées).
- **Trop grand** → peter panning (l'ombre se détache, l'objet flotte).

En pratique on ajuste à la main jusqu'à ce que l'acne disparaisse sans que le peter panning soit visible. Les moteurs exposent aussi un **normal offset bias** (décaler la position du fragment le long de sa normale) qui corrige l'acne sans autant déplacer l'ombre — souvent préféré au bias constant.

💡 Réglage pratique

1. Commencer avec un **slope-scaled bias** modéré (ex: 0.5–2.0 selon l'échelle de la scène).
2. Si acne visible, augmenter progressivement.
3. Si l'ombre se décolle du contact (peter panning), ajouter du **normal offset bias** plutôt que d'augmenter encore le depth bias.
4. Vérifier à **plusieurs distances** : le bon bias dépend de la résolution de la shadow map, qui change avec les cascades (Partie 7).

Partie 5 : Adoucir les Ombres — PCF

Le problème des ombres nettes

Le test de base est **binaire** (éclairé ou ombré). Le bord de l'ombre est donc tranché à 1 pixel — or une vraie ombre a une **pénombre** (bord flou, voir Figure 1). Une ombre parfaitement nette paraît artificielle, “jeu vidéo ancienne”. Pour la réalisme, il faut **filtrer** le test.

1. Percentage-Closer Filtering (PCF)

Principe

Au lieu d'un seul test au centre du texel, on effectue **plusieurs** tests dans un voisinage et on en fait la **moyenne**. Le résultat n'est plus 0 ou 1, mais une valeur dans $[0, 1]$ représentant la **fraction** du voisinage qui est ombrée. Au cœur de l'ombre $\rightarrow 1.0$ (tout ombré). Au bord $\rightarrow 0.3, 0.5, 0.7$ (transition douce). Loin de l'ombre $\rightarrow 0.0$.

)

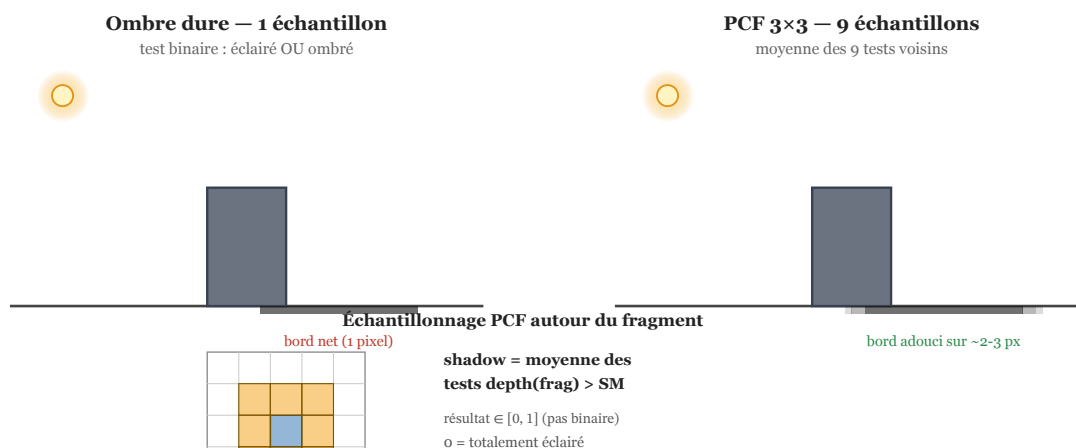


Figure 5: PCF : au lieu d'un test binaire (à gauche, bord net), on échantillonne la shadow map sur une grille $N \times N$ autour du fragment (à droite, ici $3 \times 3 = 9$ échantillons) et on moyenne les résultats. Le bord de l'ombre devient un dégradé. Plus N est grand, plus le bord est doux — mais plus le coût est élevé (N^2 échantillons par fragment).

PCF 3×3 en GLSL.

```
float pcf3x3(sampler2D shadowMap, vec3 s, vec2 texelSize) {  
    float shadow = 0.0;  
    // texelSize = vec2(1.0 / shadowMapWidth, 1.0 / shadowMapHeight)  
    for (int y = -1; y <= 1; y++) {  
        for (int x = -1; x <= 1; x++) {  
            vec2 offset = vec2(float(x), float(y)) * texelSize;  
            float stored = texture(shadowMap, s.xy + offset).r;  
            shadow += (s.z - bias > stored) ? 1.0 : 0.0;  
        }  
    }  
    return shadow / 9.0; // moyenne des 9 tests  
}
```

$\text{texelSize}(\frac{1}{\text{résolution}})$ permet de se décaler d'**exactement un texel** — sinon le filtre ne fait rien. Pour un PCF 5×5, on boucle de -2 à +2 (25 échantillons), etc.

💡 Coût et compromis

- **PCF 1×1** : 1 échantillon — ombre nette (pas de filtre).
- **PCF 3×3** : 9 échantillons — bord légèrement adouci, coût modéré. Standard “minimum” acceptable.
- **PCF 5×5** : 25 échantillons — bord bien doux, coût notable.
- **PCF 9×9** : 81 échantillons — bord très doux, coût élevé (rare en temps réel).

En pratique on utilise **poisson disk sampling** (16 échantillons répartis en cercle plutôt qu’en grille) pour un meilleur rendu à coût égal, ou le **PCF hardware** (la carte graphique fait le filtrage bilinéaire des depth samples gratuitement — donne un PCF 2×2 “gratuit”).

⚠️ PCF ne fait pas une vraie pénombre

PCF adoucit le bord, mais la **largeur** du bord est **constante** partout — elle ne dépend pas de la distance au bloqueur. Or dans la réalité, un objet près de son ombre a un bord **net**, un objet loin a un bord **très flou** (voir Figure 1). PCF ignore cette physique. C’est ce que corrige PCSS (Partie 6).

Partie 6 : PCSS — Ombres Douces Physiquement Plausibles

Percentage-Closer Soft Shadows

PCSS (Fernando et al., 2005) étend PCF pour produire une **pénombre variable** : le bord de l’ombre est d’autant plus flou que le bloqueur est **loin** du receveur. Cela imite la géométrie réelle d’une source étendue (voir Figure 1).

)

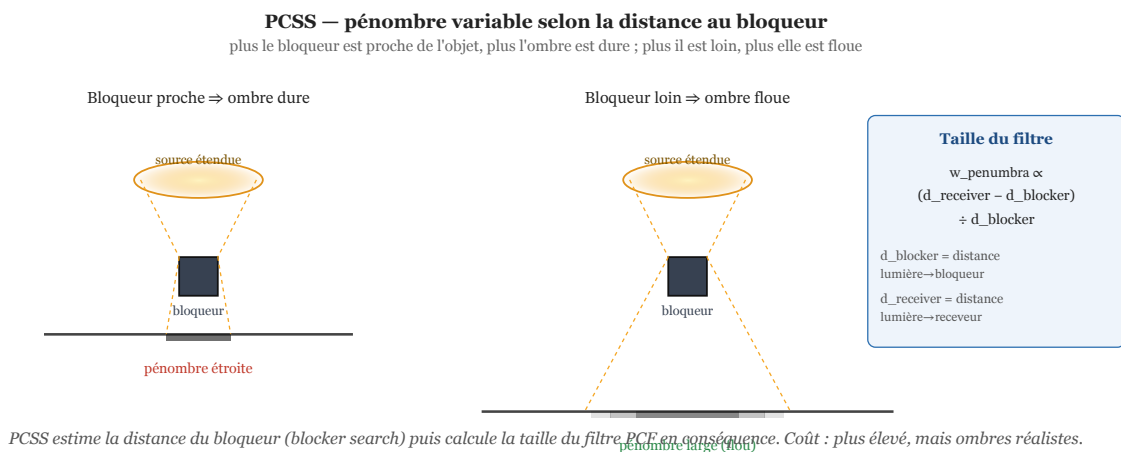


Figure 6: PCSS : la taille du filtre PCF dépend de la distance entre le bloqueur et le receveur. À gauche, le bloqueur est proche du sol → pénombre étroite (ombre dure). À droite, le bloqueur est loin du sol → pénombre large (ombre floue). C’est le comportement physique d’une source étendue.

💡 Les trois étapes de PCSS

1. **Blocker search** : échantillonner la shadow map sur une région (estimation de la **distance moyenne du bloqueur** d_{blocker}). On ne garde que les échantillons qui sont **effectivement** des bloqueurs (profondeur stockée < profondeur du receveur).

2. **Penumbra estimation** : calculer la taille du filtre w_{penumbra} proportionnelle à $(d_{\text{receiver}} - d_{\text{blocker}})/d_{\text{blocker}}$ (formule de similarité de triangles). Plus le receveur est loin derrière le bloqueur, plus le filtre est large.
3. **PCF** : effectuer un PCF avec la taille w_{penumbra} calculée — un filtre **variable** par fragment.

Détail : le blocker search

Le problème : où est le bloqueur ?

PCSS veut faire l'ombre **plus floue** quand le bloqueur est loin du receveur, et **plus nette** quand il est proche. Pour ça, il faut savoir à quelle distance est le bloqueur. Mais le shader ne connaît que la position du fragment — il ne sait pas où est l'objet qui le bloque. La seule trace du bloqueur, c'est sa profondeur stockée dans la shadow map.

La solution : regarder la shadow map autour du fragment

Pour le fragment courant, on échantillonne la shadow map sur une **petite région** autour de ses coordonnées UV — par exemple une grille 5×5 (25 texels). Pour chaque texel :

- On lit la profondeur stockée d_{stored} .
- On la compare à la profondeur du fragment (le receveur) d_{receiver} .
- Si $d_{\text{stored}} < d_{\text{receiver}}$: quelque chose est **plus proche de la lumière** que le fragment → c'est un **bloqueur**. On garde cet échantillon.
- Si $d_{\text{stored}} \geq d_{\text{receiver}}$: rien ne bloque à cet endroit → on **jette** cet échantillon.

On fait la moyenne des profondeurs des échantillons **gardés** → c'est d_{blocker} , la distance moyenne du bloqueur par rapport à la lumière.

Concret : un fragment sur le sol à 20 m de la lumière. Le fragment est sur le sol, à $d_{\text{receiver}} = 20$ m de la lumière. Un cube flotte au-dessus, à 15 m de la lumière. On échantillonne 9 texels autour de la position du fragment dans la shadow map :

Texel	Prof. stockée	< 20 (receveur) ?	Bloqueur ?	Prof. gardée
1	15	oui (15 < 20)	oui	15
2	15	oui	oui	15
3	20	non (20 = 20)	non	—
4	15	oui	oui	15
5	15	oui	oui	15
6	20	non	non	—
7	15	oui	oui	15
8	15	oui	oui	15
9	20	non	non	—

6 texels sur 9 sont des bloqueurs, tous à profondeur 15. La moyenne des profondeurs gardées : $d_{\text{blocker}} = 15$ m.

Le bloqueur est à 15 m, le receveur à 20 m → le bloqueur est à 5 m au-dessus du sol. Cette distance alimente l'étape 2 (penumbra estimation) : plus l'écart est grand, plus l'ombre sera floue.

💡 Pourquoi échantillonner une région et pas un seul texel ?

Un seul texel ne donnerait qu'un point de vue. Mais le bloqueur (un cube, un personnage) couvre **plusieurs** texels dans la shadow map. Échantillonner une région capture la **moyenne** des profondeurs du bloqueur, ce qui donne une estimation plus stable. Au bord de l'ombre — où certains texels voient le bloqueur et d'autres non — la moyenne donne une valeur de **transition**, ce qui produit un fondu progressif plutôt qu'un saut brutal.

💡 Coût et usage

PCSS est nettement plus cher que PCF (le blocker search ajoute une passe d'échantillonnage). On l'utilise pour les **ombres héros** (le personnage principal, les objets proches) plutôt que sur toutes les ombres de la scène. Unreal Engine et les moteurs modernes l'offrent en option "Soft Shadows".

Partie 7 : Cascaded Shadow Maps (CSM)

💡 CSM, PSSM — même chose

Cette technique porte deux noms selon les moteurs :

- **Cascaded Shadow Maps (CSM)** : le nom utilisé par Microsoft (DirectX), Unity, Unreal, Three.js.
- **Parallel-Split Shadow Maps (PSSM)** : le nom de l'article académique original (Fan Zhang et al., 2006), utilisé par Godot (SHADOW_PARALLEL_2_SPLITS, SHADOW_PARALLEL_4_SPLITS).

C'est exactement la même technique — le frustum est découpé en tranches, chacune avec sa propre shadow map. Dans la suite on utilise "CSM" (le terme le plus répandu dans l'industrie), mais quand vous verrez "PSSM" ou "Parallel-Splits" dans Godot, c'est la même chose.

💡 Le problème des grandes scènes

Une shadow map unique a une **résolution fixe** pour toute la scène. Pour une lumière directionnelle (soleil) qui couvre un grand terrain, 2048² étalés sur 100 m × 100 m donnent 20 texels par mètre — les ombres proches de la caméra sont **pixelisées** (crânelées), tandis que les ombres lointaines (qu'on ne voit pas) gaspillent de la résolution. On ne peut pas juste augmenter la résolution : 16K textures ne tiennent pas en VRAM.

)

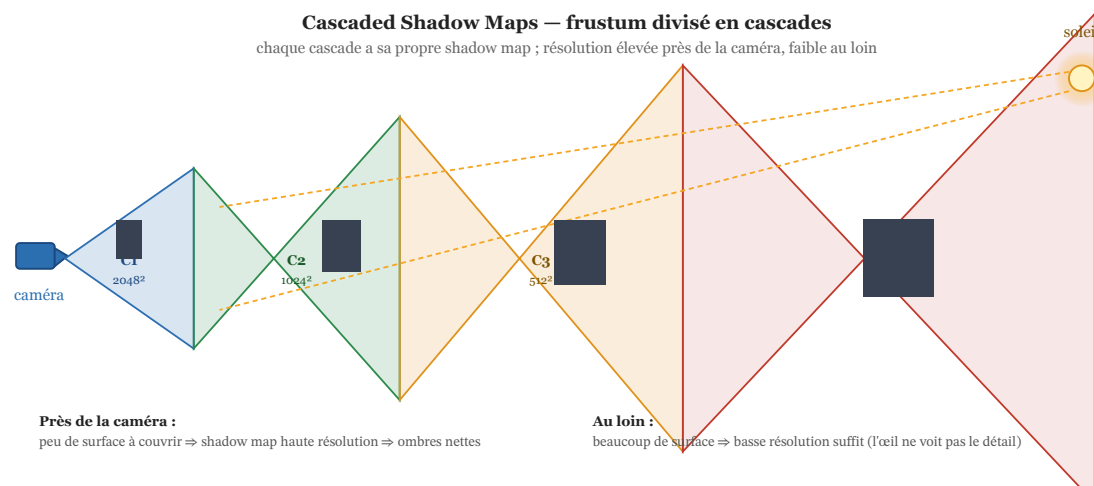


Figure 7: Cascaded Shadow Maps : le frustum de la caméra est divisé en **cascades** (ici 4). Chaque cascade a sa propre shadow map — haute résolution pour la cascade proche (C1, 2048^2), basse résolution pour la cascade lointaine (C4, 256^2). L'œil voit des ombres nettes près de lui, là où ça compte ; les ombres lointaines sont floues mais personne ne le remarque.

Principe

On découpe le frustum de la caméra en **tranches** de profondeur (les cascades), typiquement 4. Chaque cascade obtient sa propre shadow map, calculée pour **cette tranche uniquement** :

- **Cascade 1** (proche) : couvre 0–10 m, shadow map $2048^2 \rightarrow$ haute densité de texels, ombres nettes.
- **Cascade 2** : 10–30 m, 1024^2 .
- **Cascade 3** : 30–80 m, 512^2 .
- **Cascade 4** (lointaine) : 80–200 m, $256^2 \rightarrow$ basse densité, mais l'œil ne voit pas le détail.

Pendant le rendu, on choisit la cascade selon la profondeur du fragment (sa distance à la caméra). Les cascades se **chevauchent** légèrement avec un fondu pour éviter une coupure visible.

🔧 Réglages clés

- **Nombre de cascades** : 1 (pas de CSM), 2, 4 (standard), ou 8 (cinématique). Plus de cascades = plus de rendus (coût), mais meilleure qualité.
- **Split scheme** : comment découper le frustum. **Logarithmique** (plus de cascades près de la caméra) ou **practical split** (compromis logarithmique + linéaire). Le practical split est le standard.
- **Stable cascades** : on fixe la shadow map à la taille du **texel** pour éviter que l'ombre “tremble” quand la caméra bouge (sinon les texels glissent et l'ombre scintille).

⚠️ Coût du CSM

Avec 4 cascades, la passe 1 rend la scène **4 fois** (une par cascade) — c'est $4\times$ le coût d'une shadow map unique. C'est le prix des ombres de qualité pour les scènes extérieures. En intérieur (petite scène), une shadow map unique suffit et CSM est inutile.

Partie 8 : Ombres pour Lumières Ponctuelles — Cube Shadow Maps

! Le problème des PointLight

Une lumière directionnelle a une **direction unique** — une shadow map 2D suffit. Mais un **PointLight** éclaire dans **toutes les directions** (Session 5). Pour capturer “ce que la lumière voit” dans toutes les directions, il faut 6 shadow maps — une par face d’un cube centré sur la lumière. C’est l’équivalent d’un **cube map** (Session 6) mais en profondeur.

)

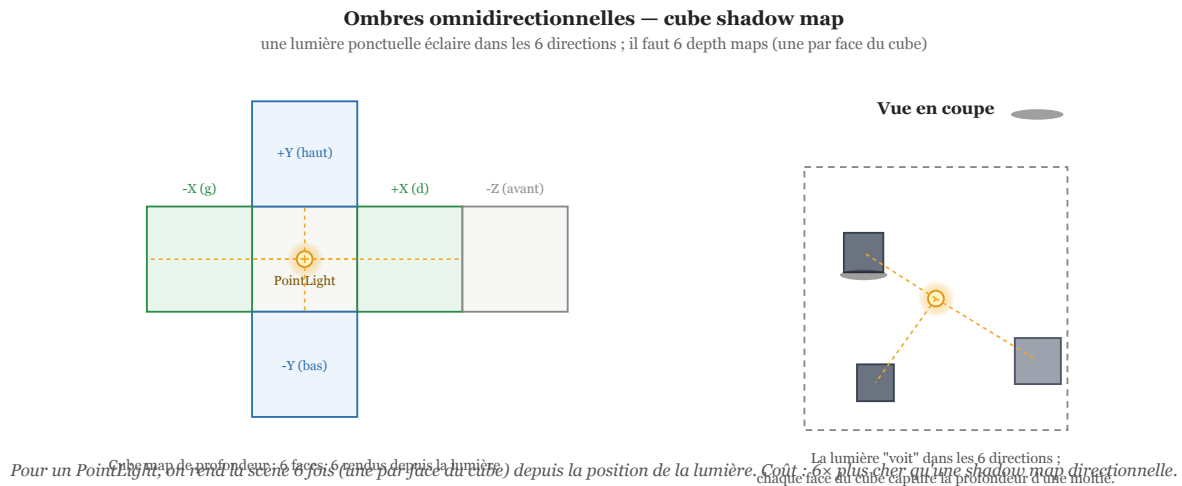


Figure 8: Ombres omnidirectionnelles : un PointLight nécessite 6 depth maps (une par face du cube). À gauche, le cube déplié avec les 6 faces. À droite, vue en coupe : la lumière “voit” dans les 6 directions et capture la profondeur de tout ce qui l’entoure.

Implémentation

Deux approches :

- **6 passes** : on rend la scène 6 fois, une par face, avec 6 view matrices ($\pm X, \pm Y, \pm Z$) et la même projection (90° FOV). Simple mais coûteux (6 rendus de scène).
- **Geometry shader / layered rendering** : une seule passe, le geometry shader duplique les triangles vers les 6 faces d’une **cube depth texture** (WebGL2 / WebGPU). Plus efficace mais moins supporté.

Pendant la passe 2, le fragment calcule la **direction** de la lumière vers lui, échantillonne le cube map avec cette direction, et compare la profondeur (ici la **distance** $|l - p|$, pas la profondeur Z).

Échantillonnage d'une cube shadow map.

```
uniform samplerCube uPointShadowMap;    // cube depth texture
uniform vec3 uLightPos;
in vec3 vWorldPos;

float pointShadowTest(vec3 worldPos) {
    vec3 toFrag = worldPos - uLightPos;    // direction lumière → fragment
    float dist = length(toFrag);           // distance réelle
    float stored = texture(uPointShadowMap, toFrag).r; // profondeur stockée
    // NB : on stocke souvent la *distance* (pas le Z) dans une cube map
    return (dist - bias > stored) ? 0.0 : 1.0;
}
```

La comparaison utilise la **distance** (longueur du vecteur) plutôt que la profondeur Z, car chaque face du cube a sa propre orientation — il n’y a pas de Z commun.

⚠ Coût et limitation

Un **PointLight** avec ombres coûte 6× une lumière directionnelle avec ombres. C’est pourquoi on limite sévèrement le nombre de **PointLight** projetant des ombres (souvent 1 seule par scène, parfois 0). Les **SpotLight** sont préférés : ils n’ont qu’une direction, donc une seule shadow map 2D — même coût qu’une lumière directionnelle.

Partie 9 : Implémentation en Three.js

Le modèle de Three.js

Three.js (utilisé depuis la Session 8) gère les ombres via trois propriétés :

- **Renderer** : `renderer.shadowMap.enabled = true` active le shadow mapping globalement. `renderer.shadowMap.type` choisit le filtre par défaut (`BasicShadowMap`, `PCFShadowMap`, `PCFSOFTShadowMap`, `VSMShadowMap`).
- **Lumière** : `light.castShadow = true` active la passe 1 pour cette lumière. `light.shadow.mapSize` règle la résolution, `light.shadow.bias` et `light.shadow.normalBias` corrigent l’acne.
- **Mesh** : `mesh.castShadow = true` fait projeter l’objet (passe 1), `mesh.receiveShadow = true` fait recevoir l’ombre (passe 2). Un mesh peut être les deux.

Contrairement à Babylon.js (qui a un `ShadowGenerator` séparé), Three.js intègre les ombres **directement** dans les lumières et les matériaux — pas d’objet supplémentaire à créer.

)

Ombres de base en Three.js.

```
// 1. Renderer – active les shadow maps globalement
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap; // filtre PCF doux par défaut

// 2. Une lumière directionnelle (le soleil)
const sun = new THREE.DirectionalLight(0xffff4e0, 2.0);
sun.position.set(10, 15, 8);
sun.castShadow = true; // active la passe 1
sun.shadow.mapSize.set(2048, 2048); // résolution de la shadow map
sun.shadow.camera.left = -15; // frustum de l'ombre
                             // (orthographique)
sun.shadow.camera.right = 15;
sun.shadow.camera.top = 15;
sun.shadow.camera.bottom = -15;
sun.shadow.camera.far = 60;
sun.shadow.bias = 0.0005; // corrige l'acne
sun.shadow.normalBias = 0.02; // corrige l'acne sans peter panning
scene.add(sun);

// 3. Les meshes qui projettent des ombres
cube.castShadow = true;
capsule.castShadow = true;
```

```
// 4. Les meshes qui reçoivent les ombres
ground.receiveShadow = true;
```

castShadow = true ajoute le mesh à la passe 1 (rendu depuis la lumière). receiveShadow = true active le test d'ombre dans le matériau du receveur pendant la passe 2. Un mesh peut être **les deux** (caster et receiver) — par exemple un personnage projette une ombre et s'auto-ombrage.

❗ Le frustum d'ombre

Pour une DirectionalLight, light.shadow.camera est une caméra **orthographique** — il faut régler manuellement son left/right/top/bottom/far pour couvrir la zone à ombrer. Trop petit → les ombres sont coupées au bord. Trop grand → la résolution est étalée (ombres pixelisées). C'est l'équivalent du problème que les CSM (Partie 7) viennent corriger.

Filtres d'ombre en Three.js.

```
// Les 4 modes de filtrage de Three.js :
renderer.shadowMap.type = THREE.BasicShadowMap;    // 1 échantillon – ombre dure (Partie 4)
renderer.shadowMap.type = THREE.PCFShadowMap;      // PCF 2×2 hardware (Partie 5)
renderer.shadowMap.type = THREE.PCFSoftShadowMap;  // PCF avec disque de Poisson (défaut)
renderer.shadowMap.type = THREE.VSMShadowMap;      // Variance Shadow Maps (Partie 12)
```

```
// VSM : régler le radius du flou
light.shadow.radius = 8; // force du flou gaussien sur la shadow map
```

- **BasicShadowMap** : pas de filtre — ombre nette, utile pour démo l'acne.
- **PCFShadowMap** : PCF 2×2 “gratuit” (hardware bilinear filtering des depth samples).
- **PCFSoftShadowMap** : PCF avec échantillons en disque de Poisson — bord doux, c'est le défaut.
- **VSMShadowMap** : Variance Shadow Maps (Partie 12) — la shadow map est floutée avant la comparaison, ombres très douces mais risque de **light leaking**.

Cascaded Shadow Maps en Three.js. Three.js n'a pas de CSM dans le core, mais fournit un module officiel `examples/jsm/csm/CSM.js` :

```
import { CSM } from 'jsm/csm/CSM.js';

// Désactiver l'ombre du DirectionalLight standard – CSM gère ses propres lumières
sun.castShadow = false;

const csm = new CSM({
  maxFar: 200, // distance couverte par toutes les cascades
  shadowMapSize: 2048, // résolution de chaque cascade
  shadowBias: 0.0005,
  shadowNormalBias: 0.02,
  lightDirection: new THREE.Vector3(-10, -15, -8).normalize(),
  lightColor: new THREE.Color(0xffff4e0),
  lightIntensity: 2.0,
  camera: camera, // CSM suit le frustum de la caméra
  parent: scene,
  cascades: 4, // nombre de cascades (Partie 7)
});

// Les matériaux doivent être patchés pour supporter les cascades
```

```
scene.traverse(obj => {
    if (obj.material) csm.setupMaterial(obj.material);
});

// Dans la boucle de rendu :
csm.update(); // recalcule les cascades quand la caméra bouge
```

Le CSM crée **N** `DirectionalLight` internes, chacune avec sa propre shadow map couvrant une tranche du frustum. Le `setupMaterial` injecte le code shader nécessaire pour choisir la bonne cascade par fragment.

💡 Référence Three.js

- Documentation : Three.js Shadows
- CSM addon : CSM example et CSM API
- La démo interactive `examples/session10_shadows.html` illustre tous ces concepts en direct.

Partie 10 : Ombres dans Godot et Unity — Aperçu

💡 Pourquoi parler des moteurs ?

La démo et la pratique de cette session se font en **Three.js** (code JavaScript, même framework qu'en Session 8). Mais les concepts du shadow mapping sont universels — voici comment les mêmes réglages se présentent dans Godot et Unity, au cas où vous travailleriez avec ces moteurs pour votre projet final.

Godot — `DirectionalLight3D` (PSSM)

Une `DirectionalLight3D` a une section **Shadow** dans l'inspecteur :

- **Enabled** : active le shadow mapping.
- **Shadow Mode** : `Orthogonal` (1 shadow map, pas de cascades), `Parallel 2 Splits` (PSSM à 2 cascades), `Parallel 4 Splits` (PSSM à 4 cascades — équivalent au CSM de la Partie 7).
- **Split 1/2/3** : positions des coupes entre cascades, en fraction de la **Shadow Max Distance** (ex: 0.1 = première cascade sur les 10% proches). Godot laisse le contrôle **manuel** des splits, contrairement à Three.js qui les calcule automatiquement (`practical / logarithmic`).
- **Blend Splits** : fondu entre cascades pour éviter une coupure visible (le "chevauchement" de la Partie 7).
- **Shadow Max Distance** : distance au-delà de laquelle les ombres disparaissent (la **shadow distance** de la Partie 11).
- **Fade Start** : les ombres commencent à s'estomper à cette fraction de la max distance (ex: 0.8 = fondu sur les 20% les plus lointains).
- **Shadow Depth Range** : `Stable` (pas de scintillement quand la caméra bouge) ou `Close` (précision proche).
- **Bias / Normal Bias** : réglages d'acne/peter panning (Partie 4).
- **Shadow Atlas** (Project Settings) : résolution globale des shadow maps (2048, 4096, 8192).

Godot — `OmniLight3D` / `SpotLight3D`

- `OmniLight3D` (= `PointLight`) : utilise une **cube shadow map** (6 faces). Coût élevé — limiter le nombre d'`OmniLight` avec ombres.

- SpotLight3D : shadow map 2D (une seule direction), beaucoup plus économique. **Préférer les SpotLight aux OmniLight** dès qu'on peut diriger la lumière.
- Réglage **Shadow > Bias** identique au DirectionalLight.

Unity — Light shadows

- **Directional Light** : **Shadow Type** = **Soft Shadows** active PCF (ou PCSS selon le pipeline). **Cascade Count** (1 à 4) règle le CSM. **Bias** et **Normal Bias** dans la section **Shadows**.
- **Point / Spot Light** : même principe, mais les PointLight utilisent une cube shadow map (coût 6×).
- **URP / HDRP** : les pipelines modernes exposent **Soft Shadows** (PCF) et **Contact Shadows** (ombres temps écran pour le contact proche, complément du shadow mapping).
- **Référence** : Unity Lights et Shadow Casting.

Partie 11 : Performance et Qualité

Les leviers de réglage

Le shadow mapping est l'un des postes les plus coûteux du rendu temps réel. Les leviers, du plus impactant au plus subtil :

- **Nombre de lumières avec ombres** : chaque lumière avec ombre = un rendu de scène supplémentaire (×6 pour un PointLight). C'est le levier n°1 — passer de 4 à 1 lumière ombrante double souvent le FPS.
- **Résolution des shadow maps** : $1024^2 \rightarrow 2048^2$ double la VRAM et le coût de la passe 1. 4096^2 est rare, réservé aux ombres héro.
- **Filtre** : PCF 3×3 (9 échantillons) est le standard. PCSS ou PCF 9×9 sont réservés aux plateformes haut de gamme.
- **CSM** : 4 cascades = 4× le coût d'une shadow map unique. En intérieur, désactiver le CSM (1 cascade suffit).
- **Distance d'ombre** : couper les ombres au-delà d'une distance (les objets lointains n'ont pas d'ombre). Énorme gain pour les scènes extérieures.

Optimisations courantes

- **Shadow distance / fade** : les ombres s'estompent puis disparaissent au-delà d'une distance. L'œil ne voit pas une ombre à 200 m.
- **Only cast, not receive** : un petit prop peut **projeter** une ombre sans jamais en **recevoir** — on économise le test d'ombre dans son matériau.
- **Static shadow casters** : pour un objet immobile, on peut précalculer sa shadow map une fois (au chargement) au lieu de la rendre chaque frame. C'est l'équivalent du baking (Session 6) mais juste pour la shadow map.
- **Frustum culling de la passe 1** : ne rendre dans la shadow map que ce qui est dans le frustum de la lumière (et qui peut projeter sur ce que la caméra voit).

Pièges fréquents

- **Pas de receiveShadows = true** : le sol ne reçoit pas l'ombre, tout paraît éclairé. Erreur n°1 des débutants.

- **Bias non réglé** : acne partout ou peter panning visible. Toujours ajuster après avoir placé la lumière.
- **Shadow map trop petite** : ombres crénelées (pixelisées). Augmenter la résolution ou utiliser le CSM.
- **Trop de lumières avec ombres** : chute de FPS. Compter les lumières ombrantes, pas les lumières totales.
- **Scintillement quand la caméra bouge** : utiliser le **stable cascades** (Godot : **Shadow Depth Range = Stable**), ou le **texel snapping** (Three.js CSM le fait automatiquement).

Partie 12 : Variante — Variance Shadow Maps (VSM)

Le principe des VSM

Les VSM (Donnelly & Lauritzen, 2006) stockent dans la shadow map non pas la profondeur seule, mais la **profondeur et le carré de la profondeur** (z et z^2). À partir de ces deux moments statistiques, on peut estimer la **variance** et appliquer l'inégalité de Chebyshev pour obtenir une **probabilité** d'être ombré — sans faire de test binaire.

Avantage : on peut **flouter** la shadow map elle-même (blur gaussien) avant la comparaison, ce qui donne des ombres douces **très** cheap (un blur post-process au lieu de N^2 échantillons par fragment).

⚠ Light leaking

Le défaut des VSM est le **light leaking** : quand deux ombres se chevauchent (un objet proche et un objet lointain alignés), la variance augmente et l'inégalité de Chebyshev "perd" l'ombre la plus lointaine — de la lumière apparaît là où il devrait y avoir de l'ombre. C'est pourquoi les VSM sont moins utilisées que le PCF/PCSS en production, malgré leur coût attractif.

Partie 13 : Démonstration en Direct — Three.js

💡 Déroulement de la démo

La démo se fait en **live** avec le fichier `examples/session10_shadows.html` — une scène Three.js interactive avec une GUI lil-gui. Un sol, un cube, une capsule animée (prouve le temps réel), une caisse, et trois lumières (directionnelle, spot, point) qu'on peut permuter. Une **preview de la shadow map** en coin bas-gauche montre ce que "voit" la lumière (Partie 2). On part d'une scène **sans ombres** et on ajoute les ombres étape par étape pour **voir** chaque défaut et sa correction.

💬 Lancer la démo

```
cd examples
python3 -m http.server 8123
# ouvrir http://localhost:8123/session10_shadows.html
```

Plan de la démo (45-60 min) — mapping avec les parties du cours

Phase 1 — Scène sans ombres (5 min) [Partie 1]

- Décocher **Lumière** → **Ombres activées** dans la GUI.

- Observer : tout est éclairé uniformément, le cube **flotte** visuellement (pas de contact), la capsule n'interagit pas avec le sol.
- Message : “sans ombres, on ne sait pas où sont les objets” — c'est le rôle perceptif des ombres.

Phase 2 — Activer les ombres de base (10 min) [Partie 2 & 3]

- Re-cocher **Ombres activées**. Observer le **contact** qui apparaît — l'effet est saisissant.
- Activer **Preview shadow map** : la depth texture apparaît en coin bas-gauche. C'est **ce que voit la lumière** (Passe 1). Montrer que les objets proches de la lumière apparaissent plus clairs (profondeur faible), les objets lointains plus sombres.
- Pointer l'**acne** sur le cube et le sol (rayures sombres) — c'est l'artefact de la Partie 4.

Phase 3 — Corriger l'acne (10 min) [Partie 4]

- Cliquer **Demos rapides** → **Montrer l'acne** : la GUI règle automatiquement bias = 0, résolution 512, filtre Basic. L'acne est **maximal** — rayures partout.
- Monter le **Depth bias** manuellement de 0 à 0.0005 : l'acne disparaît progressivement.
- Cliquer **Demos rapides** → **Montrer peter panning** : bias = 0.02, l'ombre se **détache** du cube (Peter Panning).
- Revenir à un bias modéré (0.0005) + **Normal bias** 0.02 : compromis optimal.

Phase 4 — Adoucir les bords (10 min) [Partie 5 & 12]

- Changer **Filtre : Dure (Basic)** → **PCF** → **PCF Soft** → **VSM**. Observer le bord de l'ombre qui passe de tranché à doux.
- **Basic** = “jeu vidéo PS2” (Partie 4, test binaire). **PCF Soft** = le standard moderne (Partie 5). **VSM** = Variance Shadow Maps (Partie 12) — très doux mais peut avoir du light leaking.
- Changer **Résolution** : 512 → 1024 → 2048 → 4096. Observer le crênelage qui disparaît, et le FPS qui chute. La preview de la shadow map montre aussi la différence de netteté.

Phase 5 — CSM pour une grande scène (10 min) [Partie 7]

- Changer **Taille du sol** → **Grande (200×200)**. Les cubes lointains apparaissent.
- Sans CSM : les ombres proches sont pixelisées (2048² étalés sur 200 m = 10 texels/m).
- Cocher **Cascaded Shadow Maps** → **Activer CSM** (4 cascades). Les ombres proches deviennent nettes, les lointaines restent acceptables.
- Changer **Nb cascades** : 2 → 4 → 5. Observer la qualité et le FPS.
- Montrer que la shadow map preview ne montre plus qu'une seule cascade (la plus proche).

Phase 6 — PointLight vs SpotLight (5-10 min) [Partie 8 & 11]

- Changer **Type** → **Point (ampoule)**. Observer les ombres dans **toutes les directions** (cube, capsule, caisse projettent vers la lumière centrale). La shadow map preview montre une face de la cube map.
- Noter le FPS (coin bas-droit) — chute notable (×6 sur la passe 1).
- Changer **Type** → **Spot (projecteur)** : même qualité d'ombre directionnelle, FPS remonte (1 seule shadow map 2D).
- Message : **préférer les SpotLight aux PointLight** dès qu'on peut diriger la lumière.

⚠ Points à insister pendant la démo

- **L'effet “wow” du contact** : la première fois qu'on active les ombres, la scène “prend vie”. C'est le message pédagogique principal — les ombres ne sont pas un détail, elles sont **structurantes**.
- **La shadow map preview** : c'est le pont visuel avec la Partie 2. Les étudiants **voient** ce que la lumière voit — c'est le déclic pour comprendre le shadow mapping.

- **Le compromis bias** : le faire vivre en direct (acne → peter panning → compromis). C’est l’erreur n°1 des étudiants.
- **Le coût des PointLight** : montrer la chute de FPS. Les étudiants veulent toujours mettre des PointLight avec ombres partout.
- **La shadow distance** : un levier de perf sous-utilisé. Les ombres lointaines ne servent à rien.

Partie 14 : Pratique — Ajouter des Ombres à une Scène Three.js

💡 Objectif de la pratique

Repartir de la démo `examples/session10_shadows.html` (ou d’une scène Three.js existante des sessions précédentes) et y ajouter des ombres temps réelles de qualité. On code en **Three.js** — même framework qu’en Session 8.

💬 Ce qu'on réutilise

- **Session 5** : la scène avec lumières et matériaux Blinn-Phong.
- **Session 8** : le concept de render target (la shadow map en est un), et le framework Three.js.
- **Session 6** : la comparaison avec les ombres baked (lightmap) — ici tout est temps réel, donc dynamique.

Cahier des charges

La scène à ombrer

Une scène Three.js avec :

- **Un sol** (grand, `receiveShadow = true`).
- **Plusieurs objets** projetant des ombres (`castShadow = true`) : un cube, une capsule (personnage), un ou deux props.
- **Une lumière directionnelle** (le soleil) avec ombres — utiliser le CSM pour les grandes scènes.
- **Une lumière spot** secondaire avec ombres (lampe d’ambiance).
- **Un objet en mouvement** (la capsule qui tourne ou se déplace) pour vérifier que les ombres sont bien dynamiques.

Étapes — Three.js

Workflow Three.js.

1. Charger la scène existante (ou copier `session10_shadows.html` comme point de départ).
2. Activer les shadow maps sur le renderer : `renderer.shadowMap.enabled = true;`
`renderer.shadowMap.type = THREE.PCFSoftShadowMap;`
3. Ajouter une `DirectionalLight`, régler `castShadow = true`, `shadow.mapSize (2048)`, le frustum `shadow.camera.left/right/top/bottom`, `shadow.bias` et `shadow.normalBias`.
4. Mettre `castShadow = true` sur tous les meshes qui projettent ; `receiveShadow = true` sur le sol et les murs.
5. Ajouter une `SpotLight` secondaire avec `castShadow = true` et sa propre `shadow.mapSize (1024)` — Three.js gère plusieurs lumières ombrantes indépendamment.
6. Animer la capsule (rotation ou translation dans la boucle de rendu), observer l’ombre qui suit en temps réel.
7. Pour les grandes scènes : importer `CSM from js/csm/CSM.js`, créer l’instance avec 4 cascades, appeler `csm.setupMaterial(mat)` sur chaque matériau et `csm.update()` dans la boucle.

8. **Optionnel** : afficher la shadow map en coin (comme la démo) pour visualiser la passe 1.

Étapes — Godot (alternative)

Workflow Godot.

1. Ouvrir la scène, sélectionner la `DirectionalLight3D`.
2. Section **Light** > **Shadow** > cocher **Enabled**. Choisir **Shadow Mode** = **Parallel 4 Splits** (PSSM = CSM, Partie 7).
3. Régler **Bias** (0.1 par défaut, ajuster si acne) et **Normal Bias**.
4. Vérifier que les meshes ont **Cast Shadow** = **On** (dans **GeometryInstance3D** > **Geometry** > **Cast Shadow**). Le sol doit être **On** aussi pour **recevoir** (en Godot, un mesh peut caster et recevoir simultanément).
5. Ajouter une `SpotLight3D`, activer son ombres. Comparer le FPS avec une `OmniLight3D` équivalente — montrer la différence de coût.
6. **Project Settings** > **Rendering** > **Shadow Atlas** : régler la résolution globale (2048 ou 4096).
7. Lancer, déplacer la capsule, observer les ombres dynamiques.

Critères de réussite

Ce qui est évalué

- **Contact visible** : l'ombre est attachée à la base de chaque objet — pas de peter panning.
- **Pas d'acne** : aucune rayure sombre sur les surfaces éclairées.
- **Bord adouci** : le bord de l'ombre est un dégradé (PCF activé), pas une coupure nette.
- **Ombres dynamiques** : quand la capsule bouge, son ombre suit en temps réel — preuve que ce n'est pas du baked.
- **Performance** : la scène reste à 60 FPS avec les ombres activées. Si chute, réduire le nombre de lumières ombrantes ou la résolution des shadow maps.
- **Deux sources** : au moins une ombre directionnelle (CSM) et une ombre spot, pour montrer qu'on maîtrise les deux types.

⚠ Pièges fréquents en pratique

- **Ombre invisible** : oubli de `receiveShadow = true` (Three.js) ou **Cast Shadow** = **Off** sur le sol (Godot). L'ombre est calculée mais jamais affichée.
- **Ombre invisible (Three.js)** : oubli de `renderer.shadowMap.enabled = true`. Sans ça, rien ne se passe — aucune erreur, juste pas d'ombre.
- **Acne sur tout le sol** : bias trop bas. Monter le bias **progressivement** jusqu'à disparition.
- **Ombre qui clignote quand la caméra bouge** : utiliser le CSM (qui fait le texel snapping automatique) ou vérifier la stabilité du frustum d'ombre.
- **FPS qui s'effondre** : trop de lumières avec ombres, ou résolution de shadow map trop haute. Commencer à 1024 et monter.
- **Ombre coupée au bord** : le frustum de la lumière est trop petit. Élargir `shadow.camera.left/right/top/bottom` (Three.js) ou la **shadow far** (Godot).

💡 Bonus

- **Comparaison baked vs temps réel** : reprendre la scène de la Session 9 (lightmap) et la même scène en ombres temps réel. Comparer la qualité (le baked gagne en douceur) et le FPS (le baked gagne en perf) — mais le temps réel gagne en **dynamisme** (objets en mouvement).

- **VSM** : essayer `renderer.shadowMap.type = THREE.VSMShadowMap` (Partie 12) et régler `light.shadow.radius`. Observer le **light leaking** quand deux ombres se chevauchent.
- **Contact shadows** : ajouter un effet SSAO (Partie 6 de la Session 6) pour renforcer le contact proche, en complément du shadow mapping.

Conclusion

💡 Récapitulatif

Le **shadow mapping** est la technique universelle des ombres temps réel, fondée sur une idée simple — **rendre la scène depuis la lumière pour savoir ce qu'elle voit**. Ses difficultés sont toutes dans les **détails** :

- L'**acne** et le **peter panning** (compromis de bias).
- Le **bord net** (corrigé par PCF, puis PCSS pour la pénombre variable).
- La **résolution** sur les grandes scènes (corrigée par les **cascades** CSM).
- Le **coût des lumières ponctuelles** (cube shadow maps, 6× plus chères).

Combiné avec le **baking** (Session 6) pour la géométrie statique, le shadow mapping temps réel pour les objets dynamiques donne la stratégie d'éclairage moderne : **statique précalculé, dynamique calculé**. C'est ce que font tous les moteurs professionnels — Three.js, Unity, Unreal, Godot — et c'est ce qu'on met en place dans la pratique de cette session avec la démo `examples/session10_shadows.html`.